

Agent Evolve – Product Design Specification

Product Vision

Agent Evolve is a general-purpose AI agent platform that continuously **self-improves** over time, bridging the gap between powerful but static AI models and dynamic, adaptive agents. Today's large language model (LLM) systems are often deployed with fixed prompts and configurations that **remain static after deployment**, limiting their ability to adapt to new tasks or changing environments ¹. Agent Evolve's vision is to overcome this limitation by enabling AI agents to **adaptively reason, act, and evolve in real time** ² ³. This means general users can utilize AI that learns from interactions, feedback, and experience – continuously enhancing its capabilities without constant manual reprogramming. Ultimately, the goal is to democratize advanced **self-evolving AI techniques** so non-experts can harness an AI that **improves itself** in diverse domains, from coding and mathematics to education and personal productivity ⁴ ⁵. By providing an intuitive interface on top of cutting-edge research (DeepMind's *AlphaEvolve*, Alibaba's *AgentEvolver*, UIUC's *Multi-Agent Evolve*, Princeton's *ALITA-G*, etc.), Agent Evolve strives to make adaptive, lifelong-learning agent systems accessible for everyday problem-solving ⁶ ⁷.

User Personas

- **Non-Technical Professionals** – *Knowledge workers, educators, or creatives* who want an AI assistant that can **learn on the job**. For example, a teacher could use Agent Evolve to create a tutoring agent that **generates questions for students and improves its explanations** over time via feedback, using self-evolving techniques instead of manual prompt tuning ⁸. These users need a *no-code*, friendly UI to configure tasks and view improvements.
- **Developers and Data Scientists** – *Programmers or technical users* who want to leverage self-evolving agents for tasks like **code generation and optimization**. They may use Agent Evolve to solve complex coding challenges by running an **evolutionary loop** that generates and tests code variants (inspired by DeepMind's *AlphaEvolve*) ⁶ ⁹. They benefit from being able to export **optimized code** or agent policies and inspect how the agent learned (e.g. viewing the best evolved algorithm that the agent discovered).
- **AI Researchers/Enthusiasts** – *Advanced users or researchers* interested in experimenting with novel self-evolution strategies without building everything from scratch. Agent Evolve's modular design lets them plug in custom evaluators or evolution algorithms. For instance, they might configure a **multi-agent co-evolution** scenario (Proposer-Solver-Judge triad from *Multi-Agent Evolve* ¹⁰) and visualize how agents collectively improve reasoning. These users value flexibility, extensibility, and detailed evaluation reports to analyze agent behavior.

Key Features

- **Multiple Self-Evolution Strategies in One Tool**: Agent Evolve supports a broad range of **evolutionary AI methods** out of the box, allowing users to choose or combine strategies best suited to their problem. It can perform **prompt optimization** (iteratively refining the AI's instructions for better outcomes) ¹¹, **memory enhancement** (improving what the agent

remembers or forgets each run), **tool use adaptation** (adding or tuning external tool-using skills), **workflow refinement**, and even **multi-agent co-evolution**. This breadth of methods, drawn from recent literature, ensures the agent can evolve **any aspect of its intelligence** – from its prompting and reasoning style to its available skills or team coordination ¹² ³ .

- **Self-Questioning Task Generation:** Inspired by Alibaba's *AgentEvolver*, the system features an automatic task generation module (the agent's **"self-questioning" ability**) that makes the agent proactively curious ⁸ . **The agent will explore its environment or dataset and formulate new questions or tasks for itself, rather than waiting for user instructions alone. This means a user can deploy the agent in a domain (e.g. a file system or knowledge base) and the agent will autonomously create diverse tasks** to practice and improve, eliminating the need for the user to anticipate every scenario** ⁸ . This mechanism dramatically reduces manual data or prompt engineering by letting the agent expand its own training curriculum.
- **Feedback-Driven Learning Loops:** Agent Evolve uses **experience-guided feedback loops** to improve policy and behavior, embodying the *self-navigating* principle from AgentEvolver ⁸ . After each task attempt, the agent gathers feedback – either through **automated evaluators** (test cases, scoring functions) or user ratings – and uses that to adjust its strategy. It can summarize and reuse experiences across tasks to avoid repeating mistakes, essentially learning a better **navigation strategy through its solution space** over time ¹³ . This could manifest as, for example, an agent noticing which of its answers were rated highly and updating its approach to mimic those successes (a form of reinforcement learning or fine-tuning with self-generated data).
- **Fine-Grained Trajectory Analysis (Self-Attribution):** For tasks involving long sequences or reasoning chains, Agent Evolve offers **trajectory analysis** to pinpoint which steps contributed most to success or failure. This aligns with *self-attributing credit assignment* techniques ¹³ . The agent can process an entire reasoning trace or action sequence and assign credit to intermediate steps (using methods like ADCA – Attribution-based Credit Assignment). This fine-grained reward attribution allows more efficient policy optimization: the agent doesn't just get a binary success/failure signal, but learns *which part of its chain-of-thought was effective*. In practice, users will see that the agent can debug its reasoning – e.g. "Step 3 in the solution approach was flawed, let's adjust that" – rather than treating each attempt as a black box.
- **Evolutionary Code and Algorithm Solver:** Agent Evolve includes a powerful evolutionary loop for **algorithmic problem solving**, drawing on DeepMind's *AlphaEvolve*. In this mode, the agent uses an ensemble of AI models to **propose candidate solutions in code**, then automatically runs and evaluates them against tests or objectives ⁶ ¹⁴ . A built-in evolutionary algorithm **selects the most promising solutions** to form the basis of the next generation ¹⁵ . Over successive iterations, solutions become increasingly refined. This is ideal for users who want the agent to discover efficient code or algorithms – for example, generating an optimized scheduling algorithm or solving a math puzzle with verified proofs. The agent outputs the best code it found (along with an evaluation report of its performance) at the end of the run. This feature essentially provides **AlphaEvolve-like capabilities** for general users, turning high-level problems into a guided search for high-quality code.
- **Multi-Agent Co-Evolution and Collaboration:** The product supports deploying **multiple agents that evolve together**, rather than a single monolith. Users can spin up roles such as a **Proposer, Solver, and Judge** that interact in a loop, inspired by the *Multi-Agent Evolve* framework ¹⁰ . For example, the Proposer agent might generate new challenges or test cases, the Solver agent attempts solutions, and the Judge agent evaluates both the questions and answers, providing

scores ¹⁶. All agents share an underlying model (or can have distinct models) and learn from the Judge's feedback via a joint optimization (using techniques like cooperative reinforcement learning or Task-Relative REINFORCE++ as in the UIUC research) ¹⁷. This **proposer-solver-judge triad** allows the system to improve its reasoning and problem-solving ability continuously without external data – the agents effectively **train each other** through competition and collaboration. Users can monitor this process via visualizations of how questions get harder or how the solver's performance improves over time. This feature is particularly useful for tasks like Q&A generation, complex reasoning (the agents can pose and solve new reasoning problems), and simulating adversarial training scenarios.

- **Domain Specialization via Tool Evolution (MCP):** When users need the agent to become an expert in a specific domain, Agent Evolve leverages insights from *ALITA-G*. It can perform **domain transformation** by systematically generating and curating new **tools or skills** for that domain using *Model Context Protocols (MCPs)* ¹⁸. In practical terms, a generalist agent will tackle a suite of target-domain tasks (provided by the user or via self-questioning) and identify useful strategies or subroutines that led to success. The agent then **abstracts these successful behaviors into reusable tools**, encapsulating them as MCP components ¹⁹. These might be new functions, domain-specific heuristics, or API call sequences. The collection of tools is stored in an **"MCP Box"**, essentially a plugin toolkit the agent can draw from in future tasks ¹⁹. At runtime, Agent Evolve uses retrieval methods to pick the most relevant domain tool from the toolbox for a new task (similar to retrieval-augmented generation) ²⁰. This means over time the agent **transforms from a generalist into a domain specialist**, reusing its learned tools to achieve higher accuracy and efficiency on domain tasks ²¹. For the user, this process is mostly behind the scenes – they will notice the agent getting better and perhaps see new "skills" listed in the interface as it learns them. This feature is crucial for applications like biomedical research or finance, where an agent must **gradually acquire domain-specific capabilities** beyond its initial knowledge.
- **Intuitive UI with Interactive Visualizations:** Despite the sophisticated techniques under the hood, Agent Evolve is designed with a **simple, intuitive user interface** for non-experts. The UI provides a **dashboard** view of the agent's evolution process. Key performance indicators (e.g. success rate, reward score, or accuracy) are displayed in real-time charts so users can **watch the improvement curve** as the agent evolves. Interactive visualization components allow users to drill down into details: for instance, a timeline view shows each generation or iteration, with annotations for what changes were made (e.g. prompt tweaks, new tool added, parameter mutated). Users can click on any iteration to see an "evaluation report" – a summary of outputs and the system's internal evaluation (scores from judges, test results, etc.). The UI also highlights notable **milestones** such as "achieved new best score" or "discovered new tool X," making the evolution process more transparent and engaging. Crucially, the interface abstracts away complex terminology; a non-technical user might see a button for "Improve this agent" and a dropdown of goals (accuracy, speed, creativity, etc.), rather than having to specify algorithmic details. The focus is on a **guided, gamified experience** where the user can iteratively refine what they want (e.g. "make fewer mistakes" or "be more efficient") and watch the agent autonomously attempt improvements.
- **Modular & Extensible Architecture:** Agent Evolve is built with a **modular design** that supports plug-ins and extensions at multiple levels. All core components – the agent's brain (LLM), memory store, tool interfaces, evaluator mechanisms, and evolution algorithms – are **decoupled and interchangeable** ²². Advanced users or third-party developers can extend the platform by adding new modules. For example, one could plug in a new *evaluation metric* (perhaps a custom fitness function for a novel task), a new *evolutionary strategy* (like replacing the default genetic

algorithm with CMA-ES or a reinforcement learning optimizer), or even a new *specialized agent module* (like a vision module for image-based tasks). The system exposes a **plugin API** for these extensions, and the UI allows managing installed plugins. This modularity also means Agent Evolve can integrate external environments or simulators. By providing standardized interfaces for environments and tools ²², the product can connect to things like web browsers (for an agent that evolves its web browsing skills) or game environments (for an agent learning strategies in a simulation). The architecture is future-proof – as new research emerges, users or the product team can incorporate new self-evolution techniques by adding modules rather than overhauling the system. This plug-in system ensures that Agent Evolve remains on the cutting edge of AI agent capabilities and can be tailored to niche use cases.

Architecture Overview

Figure: Conceptual architecture of a self-evolving agent system, showing how the Agent System (with its model, tools, memory, etc.) interacts with external inputs and an environment, while an Optimizer module iteratively refines the agent based on feedback loops. This design aligns with the framework described in recent surveys ³ ¹¹, covering what can evolve (prompts, memory, tools, architecture) and how the evolution is guided (evaluations, rewards, and search algorithms).

Internally, Agent Evolve’s architecture is organized into several core modules that work in a dataflow pipeline to facilitate continuous improvement:

- **Agent Core (LLM and Controllers):** At the heart is the AI model (or models) that drives the agent’s reasoning and behavior. This is typically an LLM capable of instructions and tool use. The Agent Core includes a *Context Manager* that handles multi-turn interactions and prompt assembly, and a *Controller* that orchestrates the agent’s action loop (deciding when to invoke tools, when to ask a question, etc.). The core is designed to be **model-agnostic** – users might plug in an open-source 7B model or use an API to a larger model – and the system can even use an ensemble of models for different roles (as in AlphaEvolve’s use of both a fast model and a powerful model in tandem) ⁹.
- **Memory Module:** This component manages the agent’s dynamic memory – storing past dialogues, experiences, and distilled knowledge the agent gains. It can include short-term memory (recent conversational context) and long-term memory (a vector database of important facts learned, or a summary of past outcomes). During evolution, the memory module is itself subject to optimization. For instance, the system might learn which pieces of memory to prioritize or when to purge irrelevant information, thereby evolving an optimal memory retention strategy. This design draws on ideas like gist memory for long contexts ²³. The memory module exposes an interface for the Agent Core to retrieve relevant info at each step (e.g., using similarity search to recall relevant past cases when a new task appears).
- **Tool Interface & Environment Sandbox:** Agent Evolve features an environment sandbox where the agent can perform actions and receive observations. This could be a simulated world (for an embodied agent), a set of APIs (for a tool-using agent), or just the operating system (for an agent manipulating files or browsing web pages). A standardized **Tool Interface** lets the agent invoke external tools or APIs in a controlled manner ²². For example, the agent might have a calculator tool, a web search tool, or domain-specific tools (which can be added via the plugin system). The environment sandbox captures the outcomes of these tool actions (e.g., the result of a database query) and feeds it back to the Agent Core. Crucially, the sandbox is used during evolution both

as a playground for the agent to try different strategies and as a source of evaluation data (for instance, success/failure of tasks is determined by environment feedback).

- **Evolutionary Optimizer:** This is the module responsible for orchestrating the self-improvement loop. It implements various **evolution algorithms** – such as genetic algorithms, reinforcement learning, or other search methods – that guide how the agent changes over iterations. The Optimizer takes as input the **evaluations** from each run of the agent (e.g., a reward score, success metric, or judge’s rating) and decides how to tweak the agent to improve that score. Tweaks can include *prompt mutations* (altering the agent’s instructions), *policy updates* (adjusting model weights or selecting a different LLM from an ensemble), *toolset changes* (enabling a new tool), or *architecture changes* (like splitting the agent into sub-agents or merging roles). The evolutionary optimizer may run many trials in parallel – for example, spawn 10 agent variants with slightly different prompts if doing prompt evolution, as was done in aiXplain’s Evolver system ²⁴ ²⁵. It then uses an **evaluation strategy** (like “survival of the fittest”) to pick the best variant. Over multiple generations, this yields a steadily improving agent policy. The Optimizer is extensible: one can configure it to use RL algorithms (e.g. PPO for fine-tuning the model with feedback) or population-based algorithms (e.g. genetic programming for code). In the case of multi-agent scenarios, the optimizer coordinates updates across agents, for instance using **Task-Relative REINFORCE++** to update the shared model of Proposer/Solver/Judge triad ¹⁷ or other multi-agent credit assignment techniques.
- **Evaluator (Probing & Scoring):** To close the loop, the architecture includes an Evaluator module that measures each trial’s performance. This can be as simple as a set of unit tests for code generation or as complex as an AI “judge” LLM scoring solution quality ¹⁷. In many cases, the evaluator is an automated function providing a numeric reward (e.g., number of test cases passed, or a rating of answer correctness). In other cases, it could involve human feedback (the user can step in to rate outputs, and those ratings are ingested here). The Evaluator feeds metrics into the Optimizer which then drives the next iteration. The design allows **multiple evaluators** to be combined – for example, the system can consider both **objective metrics** (tests, constraints satisfaction) and **subjective metrics** (like fluency or user preference scored by an LLM or user) simultaneously, weighted according to user settings ¹⁴. This flexible evaluation framework is important to ensure the agent evolves in the direction the user actually cares about (e.g., if the agent is writing stories, creativity might be a metric in addition to grammatical correctness).
- **Orchestration & Dataflow:** All the above components communicate via a service-oriented architecture ²⁶. For instance, when starting an evolution cycle, the Orchestrator component (part of the Optimizer) will generate a set of candidate agent configurations (different prompts or strategies), then for each: set up the Agent Core with that configuration, execute it in the environment sandbox on the given task, collect the outcome via the Evaluator, and loop back. The architecture supports **distributed execution** – heavy experiments (like running many code solutions) can be farmed out to multiple workers or cloud instances, with results aggregated for evolution. The use of a database (or in-memory store) for **experience logging** means the agent can accumulate a repository of “experiments” and outcomes. This repository is akin to the *programs database* in AlphaEvolve which stored generated solutions and their scores ²⁷. It enables analyses like finding which mutation was most beneficial or retrieving past solutions to avoid rediscovering the same thing.
- **Modularity and Interfaces:** Each module has a clear API boundary, which is how the system achieves its plug-in extensibility. For example, the *Tool Interface* exposes methods `register_tool()` and `invoke_tool()`; the *Optimizer* exposes `suggest_variants()`

and `select_survivors()`, etc. Internally, the modules can be swapped: e.g., to use a different memory store or a different judge model, without affecting the others. This architecture was influenced by AgentEvolver's **service-oriented design**, which separated environment, experience manager, advantage processor, etc., to keep the system flexible ²⁶.

In summary, Agent Evolve's architecture can be seen as a feedback loop system where the agent's **policy and configuration are continuously tuned** via an optimizer that uses **feedback from the environment** ³. The design balances **automation** (the agent largely evolves itself once set up) with **user control** (users can inject knowledge or constraints via plugins or by adjusting which metrics matter). This modular, evolvable architecture is what allows Agent Evolve to cover the wide range of self-improvement methods in a unified platform.

UI/UX Design

The user interface of Agent Evolve is crafted to simplify complex AI evolution concepts into a **visual, interactive experience** that general users can easily navigate. Key elements of the UI/UX include:

- **Getting Started Wizard:** On first launch, users are guided by a wizard to set up their agent. Instead of asking for algorithmic parameters, the UI asks in plain language: *"What do you want your agent to do?"* and *"What goal should it improve at?"*. For example, a user might select a template like "Code Generator" or "Question-Answering Assistant" or start from scratch. They then specify goals (e.g. *"maximize code performance"* or *"improve accuracy of answers"*). The wizard hides technical details by choosing appropriate evolution strategies behind the scenes (though advanced users can opt to configure manually). This ensures a gentle onboarding, focusing on the user's objective rather than the mechanics.
- **Dashboard and Run Console:** The main screen is a **dashboard** showing the agent's current status and performance. A prominent element here is the **Evolution Progress Graph** – a live-updating chart that plots the agent's performance metric across generations/iterations. For instance, if the agent is improving code, the graph might show "test cases passed" or an efficiency score on the Y-axis over time, giving a quick sense of improvement. Alongside the graph, key statistics are listed: current best score, generation count, time elapsed, etc. Users can start, pause, or stop the evolution process with simple controls (similar to a media player, with a "play/pause" button for the evolution loop). The **Run Console** tab displays a realtime log of what the agent is doing: e.g., "Generation 5: testing 10 variants... Variant 5b passed 8/10 tests, new best solution found." This console uses easy-to-read language and occasional highlights (like green text for improvements) to keep users informed without drowning in data.
- **Evolution Settings Panel:** For users who want more control, a collapsible panel or modal allows tweaking settings. Here, users can switch on/off certain evolution methods (e.g. toggle "Prompt Optimization" or "Tool Evolution") and adjust sliders for how much the agent explores vs. exploits (akin to mutation rate or learning rate, but presented in lay terms like "Exploration: Low <-> High"). They can also choose the **output mode** here – for example, selecting whether the end product should be *"Generate optimized code"*, *"Produce a detailed report"*, or *"Interactive visualization"*. These choices correspond to the output modes described in the next section. The UI ensures safe defaults and provides tooltips (with non-jargon explanations possibly referencing real examples: *"e.g. enabling 'Multi-Agent' adds a second agent to critique and improve the first's answers"* ²⁸).

- **Visualization of Agent's Thought Process:** One innovative UI feature is a **"Thought Trail" viewer**. Since the agent often generates chain-of-thought reasoning or multiple attempts, the UI captures these and presents them in an interactive timeline or tree. For example, if the agent is solving a math problem and tries a few approaches internally, the user can click "View Thought Trail" to see a step-by-step trace of the agent's reasoning (formatted similar to a chat or flowchart). Crucially, this trail is annotated with feedback: steps that the agent's *self-attribution* module found useful are marked in green, and missteps are marked in red with notes on what was learned ¹³. This feature turns the often opaque process of AI reasoning into a tangible artifact the user can inspect, which builds trust and insight. It also ties into the educational aspect – users can learn from the agent's evolving thought patterns.
- **Multi-Agent Coordination View:** If the user is running a multi-agent evolution (with roles like proposer/solver/judge), the UI provides a **collaboration view**. This might be depicted as a triangle or circular diagram showing each agent role and arrows representing questions/answers/judgments exchanged ¹⁰. As the agents operate, the arrows might pulse or update with snippets (e.g., "Judge: score=8/10 for this answer"). There's also a leaderboard-style table if applicable, showing how each agent (or each variant in population-based runs) is performing. This gives a real-time window into the co-evolution dynamics that would otherwise be abstract. Users can literally **watch the proposer agent generate new tasks and the solver agent tackling them**, which demystifies the co-evolution process.
- **Results and Reports:** After or during the evolution, users can switch to a **Results** tab. Here, the final outputs are presented in the user's chosen format (code, policy, report, etc., see Output Modes below). If it's code, the UI shows the code with syntax highlighting and maybe an execution result summary. If it's a policy or prompt, it's displayed as a formatted text file with notes on how it evolved. Alongside the output, an **Evaluation Report** is generated, summarizing the agent's performance. This report might include a table of metrics (initial vs final performance, improvements in each criterion ²⁹), a timeline of major improvements, and recommendations for further use. The UI allows exporting this report (e.g., PDF or JSON) so users can share how their agent improved or keep a record for auditing (important for domains like finance or healthcare where you need to trace how an AI changed).
- **Interactive Visualizations:** For scenarios where an interactive visualization output is chosen (e.g., evolving a strategy that can be visualized), the UI can render custom visuals. For example, if the agent was evolving a path-finding strategy in a grid, the final interface might present a small simulator with a "play" button to watch the agent navigate the grid using its evolved policy. Or if evolving an algorithm, maybe a chart comparing the efficiency of the evolved algorithm vs standard algorithms is displayed. These visual outputs make the results more comprehensible to users who might not parse code or raw data easily.
- **Help and Guidance:** Throughout the UI, context-sensitive help is provided. A sidebar might highlight *"Behind the scenes: Agent Evolve is now trying different prompt phrasings to see which yields better results (a technique known as prompt evolution ³⁰)"* in friendly language. There's also a documentation section (possibly with a searchable FAQ or even an interactive chatbot assistant within the app) for explaining concepts like "what is self-attribution?" or "how does multi-agent evolution work?". The idea is to educate interested users so they can get the most out of the tool, but not overwhelm those who just want to use it at a high level.

Overall, the UI/UX focuses on **clarity, control, and transparency**. Agent Evolve turns what could be an esoteric process (AI self-improvement) into a **guided, observable journey** for the user. By letting users

watch and steer the evolution in an intuitive way, the product ensures a broad range of users – even those unfamiliar with AI – can confidently deploy self-evolving agents for their own needs.

Supported Evolution Methods

Agent Evolve incorporates a wide array of evolution methods identified in research, allowing it to target different aspects of an agent’s intelligence. The core supported methods include:

- **Prompt Optimization:** The system can evolve the agent’s prompts or instructions to maximize performance. This can be done via **iterative prompt editing** or using **genetic algorithms on prompts** (as in *EvoPrompt* ³¹). For example, the agent might start with a generic instruction and through evolution end up with a highly specialized prompt that yields better results. Research has shown that even LLMs themselves can be used as effective prompt engineers ³², so Agent Evolve leverages that by sometimes having one model generate improved prompts for another. Prompt optimization can occur *at training time* (refining a base prompt via reinforcement learning from evaluations) or *at test time* (dynamically adjusting prompts based on feedback). Users see this in practice when the agent’s instructions or system messages change over generations to better guide its behavior.
- **Memory Evolution:** Beyond prompt text, the agent’s memory usage can evolve. **Memory evolution** involves optimizing what the agent stores and recalls. For instance, if an agent is solving a sequence of tasks, it may learn to retain certain intermediate results or summaries that are most useful for future problems. Techniques here include **memory trimming** (dropping unhelpful memories) and **memory augmentation** (learning to construct better summaries). This category aligns with “Context Evolution” in literature, which covers memory and context window management ¹¹. A practical example is an agent reading a long document: it might evolve to note only the key points (“gist memory”) for later questions, as human readers do ²³. Agent Evolve supports memory modules that adapt in this way, and even the size or structure of memory (like how many past interactions to remember) can be subject to evolution.
- **Tool Use and Tool Creation:** Agents often rely on external tools (calculators, web search, databases). Agent Evolve can evolve both the *choice* of tools and *how* they are used. In terms of **tool use optimization**, the agent can learn which tool to invoke at what time (similar to planning tool API calls effectively) ³³. More intriguingly, following approaches like *CREATOR* and *ALITA-G*, Agent Evolve supports **tool evolution/creation** ³⁴ ¹⁸. If a needed capability is missing, the agent can actually propose a new “tool” – for example, writing a snippet of code to handle a subtask (this snippet becomes a new callable function in its toolbox). The agent then generalizes that tool for reuse. By standardizing tools via Model Context Protocol (MCP) as in *ALITA-G* ¹⁸, the system ensures any newly created tool can be easily integrated. Over time, the agent amasses a *customized toolset* that is optimized for its task distribution. This method is powerful in specialized domains: an agent working on math problems might “discover” a new solving technique and encapsulate it as a tool for future problems.
- **Workflow and Policy Adaptation:** Agent Evolve can modify the **workflow** or internal decision policy of the agent. This includes adjusting the sequence in which the agent tackles subtasks (or in a multi-agent setting, which agent handles which part). Drawing from approaches like *AFlow* (Agent Workflow generation) ³⁵ and dynamic agent networks ³⁶, the platform can evolve the agent’s strategy. For example, an initial workflow for a customer support agent might be linear (greet -> lookup info -> answer). Through feedback (maybe noticing delays or errors), the agent could evolve a branched workflow (e.g. greet -> if user is verified skip some steps -> prioritize

certain info first, etc.). In multi-agent cases, **workflow adaptation** might reassign tasks among agents or introduce new roles. Essentially, the “architecture” of the agent’s reasoning process is not fixed – it is tuned for efficiency and reliability ³⁷. Users can observe this when the agent starts to handle tasks in a different order or uses parallelism after a few iterations (implying it learned a better plan). This method overlaps with *single-agent vs multi-agent architecture optimization* as identified in surveys ¹¹ – the system can even decide to split one agent into two cooperating agents if that proves beneficial (or vice versa).

- **Co-Evolution in Multi-Agent Systems:** In scenarios with multiple agents, Agent Evolve supports **co-evolutionary methods** where agents improve together by interacting. This is akin to having a population of agents or roles that learn in tandem, sometimes competitively, sometimes cooperatively. One implemented approach is the *Proposer-Solver-Judge triad* from Multi-Agent Evolve ¹⁰. Here, the Proposer’s evolution (generating better challenges) is tied to the Solver’s evolution (solving better) and the Judge’s (evaluating more accurately). The platform can use **multi-agent reinforcement learning** and evolutionary game strategies so that, for example, two agents playing a game learn from self-play (each is the evolving environment for the other). Co-evolution ensures robustness: by having agents test each other, the system avoids overfitting to a single evaluator. Another example is evolving a team of specialized agents (as in *TeamPrompt* or evolutions in AgentVerse) where each agent’s policy is updated based on team performance. Agent Evolve allows the user to configure such setups, and under the hood uses algorithms like **population-based evolutionary strategies** (treating each agent as an individual in a population) or **cooperative co-evolution** (optimizing subsets of agents with coordinated fitness). The outcome is a **diverse but coordinated agent system** that can tackle complex tasks beyond the capability of any single agent. This method is especially useful for open-ended tasks (e.g., brainstorming creative ideas) where a solitary agent might stagnate but a group can spark novel outcomes.

- **Evolutionary Algorithm-Based Tuning:** At its core, Agent Evolve leverages classical **evolutionary algorithms** (EAs) to fine-tune agent parameters and policies. This includes genetic algorithms, evolutionary strategies, and other population-based searches. For instance, if the agent’s behavior is governed by certain numerical parameters (temperature settings, thresholds for decisions, etc.), the system can treat each set of parameters as a “genome” and evolve them. The platform integrates approaches like those in *EvoAgent* (which applied genetic algorithms for multi-agent configurations) ³⁸. Additionally, aspects of the model itself can be tuned by EAs: e.g., choosing which few-shot examples to include in context can be formulated as an evolutionary search. In code generation scenarios, the system’s approach of generating and selecting programs is essentially a form of genetic programming (where mutations are code edits and crossover can be combining parts of two solutions) ¹⁵. Agent Evolve abstracts this so a user doesn’t need to know the term “genetic algorithm” – they simply see that the system is **trying many variations and iteratively improving the best ones**, which is the essence of evolutionary tuning. By supporting EAs, the platform can escape local optima that simpler gradient-based methods might get stuck in, exploring a broad solution space to find truly novel strategies (as evidenced by AlphaEvolve discovering non-intuitive, effective solutions in data center scheduling) ³⁹.

Each of these evolution methods can be used in isolation or combined. For example, a typical run might use **prompt optimization + reward-based RL** initially, then introduce **tool creation** once the agent hits a performance plateau, and finally use **co-evolution** with a second agent to polish the results. This layered approach aligns with the idea of *lifelong learning* – different techniques kick in at different stages of the agent’s development ³ ⁴⁰. By covering the full spectrum of strategies documented in “self-evolving agent” research, Agent Evolve ensures that whatever the **bottleneck to improvement** is –

be it a suboptimal prompt, lack of knowledge, poor strategy, or insufficient coordination – the platform has a method to tackle it.

Output Modes

Agent Evolve is designed to deliver results in formats that best suit the user's goals. Upon completion (or at checkpoints), users can choose from several **output modes**:

- **1. Code Generation Output:** If the task involves creating or optimizing code (as with algorithm design or coding challenges), the system outputs the **evolved code** along with documentation. Specifically, users get the final code solution that the agent deems best, which is typically well-tested and optimized through the evolutionary loop ⁶ ¹⁴. The output isn't just raw code; Agent Evolve provides inline comments explaining any novel or non-obvious sections (especially if the code was synthesized through unconventional means). Additionally, an **execution summary** is included: e.g., "This script passes 48/50 test cases, with a 20% efficiency improvement over the baseline." The idea is to make the code immediately usable and understandable. This mode is ideal for developers who want a piece of code they can drop into their project, or for educational users to study the solution the AI evolved.
- **2. Agent Policy/Configuration File:** In cases where the user's goal was to train an agent for ongoing use (say a customer service chatbot or a game-playing agent), Agent Evolve can output an **agent policy file**. This could take the form of a saved model checkpoint, a prompt or chain-of-thought template, or a structured configuration (for example, a JSON or YAML describing the agent's workflow, tool access, and key prompt phrases). Essentially, this file encapsulates the **"brain" of the evolved agent**. A user can then deploy this policy in production or load it later to resume work. For instance, after evolving a chatbot that learned to handle tricky customer queries, the output might be a refined prompt + a set of new example dialogues (the "knowledge" it accumulated). In multi-agent setups, the policy file might detail each agent's role and refined instructions. Providing the policy in a transparent format allows users to review exactly what rules or behaviors the agent will follow – an important factor for trust and verification. It also means the improvements are not ephemeral; they persist as a tangible artifact the user owns.
- **3. Evaluation Report:** No matter which concrete output (code, policy, text) the user chooses, Agent Evolve can generate a comprehensive **evaluation report** detailing the agent's performance and evolution journey. This report includes sections like *Overview of Improvements* (e.g., "accuracy improved from 70% to 90% over 20 generations"), *Techniques Used* (listing which evolution methods were applied, like prompt tuning, tool creation, etc.), and *Before vs After Comparison*. It might show examples of the agent's output before evolution and after, to highlight the differences (for instance, an early attempt at an answer vs the final answer, with the latter being more accurate and well-formatted). The report also includes any relevant metrics and charts – for a code task, maybe a graph of runtime efficiency over iterations; for a QA task, a table of evaluation scores by the judge agent across rounds. Citations to known benchmarks or standards could be included (the system can note if the final agent surpasses a known baseline, for example). The **purpose of the evaluation report is twofold**: to give credit and insight into how the solution was achieved (useful for learning and debugging), and to provide an artifact that might be required for auditing or documentation, especially in professional settings where one must validate the AI's development process ⁴¹ ⁴². Users can export this report as a PDF or HTML file. It's written in a clear, layperson-friendly style whenever possible, explaining the agent's gains in plain language (with technical appendix available for those interested).

- **4. Interactive Visualization:** For some tasks, the best way to understand the output is to see it in action. Agent Evolve can produce interactive visualizations as output when appropriate. For example, if the task was to evolve a strategy in a board game, the output could be a small web-based viewer where one can simulate games using the evolved agent and see the moves. If the task was optimizing a neural network architecture, the output visualization might be a diagram of the network before and after, with the ability to toggle layers. In more general use, one powerful visualization is an **“evolution playback”**: a slider or animation that shows the state of the agent at each generation, so a user can literally watch how the agent improved. For instance, if evolving an image classification prompt, the visualization might cycle through heatmaps showing how the focus changed with each iteration. These interactive outputs are especially useful for presentations or educational purposes – they allow others to intuitively grasp what the AI has learned. Users can host these visualizations on the web (Agent Evolve can export them as a self-contained HTML/JavaScript bundle) or embed them in their own reports.
- **5. Natural Language Summary:** In use cases oriented toward insights rather than artifacts, the system can produce a **natural language summary** of what was accomplished. This is essentially the agent explaining its solution or a summary of improvements. For example, after evolving a complex solution, the agent might output: **“Summary:** Over 15 iterations, I learned to prioritize the user’s question type. Early on, I often gave generic answers, but by iteration 15, I started using the appropriate tool (database vs internet search) depending on context, which improved accuracy by 20%.” Such a summary is useful for stakeholders who just want the gist, not the technical details. It harnesses the LLM’s ability to generate human-readable explanations and provides a narrative of the evolution. This mode can be supplemental to the above outputs, giving an executive summary that can accompany, say, the code or policy file.

Users are not limited to choosing just one output; the UI allows selecting multiple outputs if desired. For instance, a data scientist might retrieve both the **code** and an **evaluation report** to satisfy both implementation and documentation needs. Under the hood, these outputs are generated directly from the agent’s final state and knowledge. The **code** or **policy** is the direct product of the agent’s last iteration. The **report** and **summary** are produced by analyzing logs and using the LLM to format the insights (with the judge agent or a separate reporter agent ensuring factuality). The **visualizations** are created using recorded data from the evolution process.

By providing versatile output modes, Agent Evolve makes sure that the end results of using a self-evolving agent are **tangible and actionable** for the user. Whether the user needs a ready-to-use solution (code/policy), wants to learn and trust the process (report/summary), or wishes to demonstrate the results (visualization), the platform has them covered.

Technology Stack

Agent Evolve’s implementation leverages a robust technology stack, combining proven AI frameworks with user-friendly application tech:

- **Core Language and Frameworks:** The backend is primarily built in **Python**, taking advantage of its rich ecosystem for AI research and rapid prototyping. Key libraries include **PyTorch** (for any training or fine-tuning of models during evolution) and the **Hugging Face Transformers** suite for easy integration of LLMs. The evolutionary algorithms are implemented using libraries like **DEAP (Distributed Evolutionary Algorithms in Python)** for genetic algorithms and **RLlib** (from Ray) for reinforcement learning components, where applicable. The design follows modular

principles – for example, the memory module might use a **vector database** (like FAISS or Chroma) under the hood to store embeddings of past contexts for quick retrieval.

- **LLM and Model Integration:** Agent Evolve is model-agnostic but comes pre-integrated with several LLM options. By default, it might use a moderately sized open-source model (for offline or local use) and also offer connectors to API-based models (e.g., OpenAI GPT-4 or Google's Gemini, if available) for users who have API keys. The architecture allows multiple models to be used simultaneously (as AlphaEvolve did with different Gemini models for breadth vs depth of ideas ⁹). This is facilitated by an **AI Orchestration layer** that abstracts model calls, possibly using libraries like **LangChain** to manage prompt assembly, tool usage and multi-model pipelines. The models run in a sandbox that can be scaled – for heavier tasks, the system can distribute model inference across GPUs or even spin up workers on cloud providers.
- **Front-End:** The user interface is built as a **web application** for accessibility. Technologies like **React** (with TypeScript) are used to create a dynamic, responsive UI that can run in any modern browser. This web app can be served from a local Electron wrapper for the desktop application version (packaged as a cross-platform desktop app), and also deployed to a web server for the browser-based version. The interactive charts and visualizations rely on libraries such as **D3.js** or **Plotly** for graphs, and perhaps custom Canvas/WebGL for more complex visual replays. The UI communicates with the backend via a local HTTP or WebSocket API. For example, the dashboard might open a WebSocket to stream live logs and metrics from the backend as the agent runs. This ensures the UI remains reactive to long-running backend processes.
- **Backend Server:** The core logic runs in a **Python server** (which could be started as a local service when the desktop app launches, or run as a persistent cloud service for the web version). This server exposes RESTful endpoints and socket channels for the front-end to interact (for starting an evolution run, pausing, fetching intermediate results, etc.). It uses a lightweight framework (like **FastAPI** or **Flask**) given that much of the heavy lifting is internal and asynchronous. For concurrency (like evaluating many variants in parallel), it leverages **asyncio** and multi-processing or Ray, ensuring that multiple trials can run in parallel to speed up evolution.
- **Data Storage:** The agent's state, configurations, and evolution history are stored in a combination of formats. A **lightweight database** (like SQLite or TinyDB) may store experiment configurations, user settings, and summary results. The experiences (trajectories, intermediate solutions) might be stored as JSON or pickled objects on disk for each run (to enable the "playback" feature of evolution). Larger data, like a library of generated candidate programs or an archive of failures (which aiXplain's Evolver uses to avoid repeated mistakes ⁴³), can be stored in structured directories or databases as well. If the user is running the system in a cloud setup, an object storage (like S3) could be used for these artifacts. The output artifacts (code, policy files, reports) are saved in a user-specified workspace directory for easy access.
- **Tool Integration Layer:** To allow the agent to use external tools safely, the system includes a sandboxed execution layer. For instance, if the agent wants to run generated code, it is executed in an isolated environment (using Docker containers or sandbox libraries) to capture output and errors without risking the host system. Tools like **pydide** or **Node.js sandbox** might be used for running code in various languages. Web browsing tools might rely on headless browsers or API calls (with careful monitoring to avoid uncontrolled actions). All tool usage is mediated by this layer, which logs tool inputs/outputs for the evolution process and ensures compliance with security (so a self-evolving agent can't, say, accidentally write to system files outside its allowed scope).

- **Plugin System:** The modular design allows plugins, which are essentially Python packages or scripts that adhere to a certain interface. For example, a new Evolution strategy plugin might subclass an `EvolutionStrategy` base class and implement a `next_generation(population_scores)` method. The system scans a `plugins/` directory or uses entry points to load available plugins on startup. It uses dependency injection to incorporate these into the main loop. To avoid stability issues, plugins run in their own process or are invoked carefully, and the UI will indicate if a third-party plugin is being used (for transparency). The plugin architecture is influenced by the way ML frameworks like **TensorFlow** or **PyTorch Lightning** allow callbacks and extensions.
- **Testing and Benchmarking Tools:** Under the hood, Agent Evolve includes a suite of benchmarking tools drawn from academic benchmarks (the system was tested on things like GAIA, HLE, and other tasks referenced in ALITA-G and surveys ⁴⁴ ⁵). These benchmarks are not exposed directly to end-users, but they ensure the platform's methods are validated. For users interested, an “evaluation mode” can run standard tasks to see how well the self-evolution performs relative to known results (essentially replicating experiments from *A Comprehensive Survey of Self-Evolving AI Agents* to give users confidence).
- **Deployment:** The desktop application is packaged via **Electron** (combining the React front-end and Python backend into a single bundle) or via **PyInstaller/Docker** for those who want a more devops approach. The web version can be offered as a hosted service with a scalable backend (Kubernetes or similar to spin up containers for each user's agent runs, as needed). All sensitive operations (like possibly accessing user-provided data for tasks) are kept local in the desktop version to alleviate privacy concerns, whereas the cloud version emphasizes convenience and heavy compute availability.
- **Security and Safety Mechanisms:** Given that the agent can evolve and potentially take unforeseen actions, the stack includes safety interlocks. There's a **monitor** process that watches for anomalous behavior (e.g., if a self-modified prompt tries to instruct the agent to do something harmful or if a tool action goes out of bounds). Based on AI safety research, this monitor can employ rule-based checks as well as a safety LLM that evaluates the agent's actions (aligning with the notion of ensuring safe self-evolution as highlighted in surveys ⁴⁵ ⁴⁶). If a violation is detected, the system halts the agent and alerts the user. The sandbox and least-privilege principles in the Tool layer further mitigate risks.

In summary, the technology stack of Agent Evolve marries **AI research frameworks with modern web development**, ensuring a smooth experience for the user and a reliable, extensible foundation for the system. The choices prioritize accessibility (web-based UI, cross-platform), adaptability (plugin architecture, multiple AI model support), and safety (sandboxing, monitors), in line with the product's goal of bringing advanced AI evolution to general users in a responsible way.

Roadmap

To achieve the ambitious vision of Agent Evolve, development is planned in iterative phases. Each phase expands the platform's capabilities and incorporates user feedback, as well as the latest research breakthroughs:

- **Phase 1: MVP – Single-Agent Evolution (Q1 2026)**

The initial release will focus on getting the core loop working for single-agent scenarios with a few key methods. The MVP will include prompt optimization, basic reward-based learning, and

evolutionary code generation. Users will be able to set up an agent with a goal (e.g., solve coding puzzles or answer trivia questions) and watch it improve via self-questioning and simple feedback loops. We will integrate one LLM (an open-source 7B model) and a few tools (like a Python executor for code and a calculator for math). The UI will have the dashboard, basic visualization of improvement, and allow exporting code and an evaluation report. This phase is about proving the concept that a general user can meaningfully improve an agent's performance without manual intervention. We'll validate on scenarios like those in DeepMind's AlphaEvolve blog – e.g., have the agent evolve a better sorting algorithm or a better prompt to summarize text ⁴⁷. Success in Phase 1 is measured by the agent achieving improvements comparable to published results on a couple of benchmarks (say solving a coding challenge it couldn't initially) and positive user testing feedback on the UI simplicity.

- **Phase 2: Expanded Evolution Techniques & Domain Specialization (Q2–Q3 2026)**

In this phase, we enrich the method toolkit. Memory evolution and tool evolution will be introduced, leveraging the ALITA-G concept of MCP tool generation ¹⁸. The agent will be able to invent new tools and store long-term knowledge, enabling it to tackle more complex, long-horizon tasks. We will also add the multi-modal ability to handle text+images (if relevant to domain specialization, e.g., a tool to fetch an image if needed). Phase 2 will also see the introduction of **domain packs** – pre-made configurations for certain domains (coding, math, healthcare Q&A, etc.) that come with domain-specific evaluators or tools. For example, a *Programming Pack* might include unit test generation for code (to better evaluate code quality). These packs serve as jump-start templates for users. The UI in this phase will get enhancements like the thought trail viewer and more settings for advanced users. From a research integration perspective, Phase 2 aligns with implementing findings from the surveys: we'll incorporate more of the unified framework aspects ⁴⁸, ensuring our system addresses all “what to evolve” dimensions (prompt, memory, tools, architecture). We will likely release a technical report or paper during this phase to share the inner workings and perhaps compare Agent Evolve's performance to results from *AgentEvolver* and *ALITA-G* on their respective benchmarks, demonstrating that our platform can achieve similar gains in an accessible manner ⁴⁹ ⁴⁴.

- **Phase 3: Multi-Agent Co-Evolution & Collaboration Features (Q4 2026)**

Phase 3 will introduce full **multi-agent support**. Users will be able to add multiple agents in a project, assign roles, and enable co-evolution strategies (e.g. cooperative or adversarial setups). We will implement the proposer-solver-judge loop from Multi-Agent Evolve ¹⁰ as a built-in template. Additionally, we may introduce other multi-agent patterns such as MetaGPT's collaborative agents or Camel's communicative agents ⁵⁰ ⁵¹ as optional templates, since those have shown promise in distributed problem solving. The UI will therefore include the collaboration view to monitor these interactions. This phase will also improve the **team tuning** aspect as highlighted by aiXplain's Evolver (optimizing multi-agent teamwork) ⁵². Technically, this will involve ensuring our backend can handle multiple concurrent LLMs and possibly training them via multi-agent reinforcement learning. We'll optimize performance, likely by introducing more efficient models (maybe integrate model quantization or smaller specialized models for judge roles). A key deliverable in Phase 3 is the **Plugin SDK** – clear documentation and examples for third parties to add custom evolution strategies or agent types. We might start building a community around extensions, possibly hosting a repository (inspired by the Awesome Self-Evolving Agents repo ⁵³ ⁵⁴) where users share plugins and use-cases. By the end of this phase, Agent Evolve should cover the majority of techniques identified in academic surveys ³ ¹¹, effectively becoming a practical reference implementation of self-evolving agents.

- **Phase 4: Refinement, Safety, and Scale (2027)**

After core features are in place, the focus shifts to **refinement and safety**. We will conduct

extensive testing, especially in domains like healthcare or finance, to ensure the agents evolve within safe bounds (no unintended behaviors). This may involve adding more robust guardrails and perhaps a *“judge agent” dedicated to ethical considerations* that evolves alongside (for example, evaluating not just task performance but also compliance with constraints – an idea derived from the need for safe co-evolution ⁴⁵). We will integrate user feedback to polish the UI/UX, making any complex parts more accessible. Another aspect of Phase 4 is **optimization and scalability**: improving the efficiency of the evolution loop (so it can run faster or handle larger models). We may implement distributed compute more fully, allowing power users to hook up a cluster or cloud resources for big tasks. On the UI side, we envision launching a **cloud service** version of Agent Evolve, so users who prefer not to run it locally can use a web portal (with usage billed or metered if heavy compute is needed). This would also allow easy sharing of evolved agents – a sort of “agent app store” where people can publish agents or building blocks (subject to appropriate review). Additionally, we plan formal **benchmark releases** – for example, a leaderboard for self-evolving agent performance on certain tasks, to encourage research and usage (our platform could become a standard evaluation harness given it implements many techniques).

- **Phase 5: Personal AI Ecosystems and Continuous Learning (Beyond 2027)**

Looking further ahead, Agent Evolve will move towards handling **ecosystems of agents and lifelong learning in the wild**. This is inspired by future directions in literature where multiple agents might form an evolving society or where an agent continues to learn over months/years in a changing environment ⁵⁵ ⁵⁶. We aim to support persistent agents that can be your personal AI, improving as you use them daily (for instance, an email assistant that quietly evolves better email sorting and responses over time through your feedback). Technologically, this may involve tighter integration with user data (with privacy controls) and incremental model updating (possibly leveraging on-device learning for privacy, or federated learning if multiple users’ agents learn from shared trends). Another frontier is **multimodal and real-world integration**: by Phase 5, we expect to incorporate sensors or real-time data streams so an agent can evolve with live inputs (think IoT or robotics domains). We also foresee collaboration with hardware (maybe using efficient edge AI chips for continuous learning without relying solely on cloud). Throughout, maintaining alignment and safety will be paramount – we will likely collaborate with the broader AI community on standards for evaluating and controlling self-evolving agents, contributing lessons from our user-facing deployment.

The roadmap above is subject to adapt based on user needs and emerging research. We will remain agile – for example, if a breakthrough in self-evolving agents occurs (say a new algorithm that dramatically improves sample efficiency), we will prioritize integrating that as a feature or plugin. Our plan is to update the platform frequently (with minor updates each sprint and major releases aligning with the phases). By incrementally building up capabilities, we ensure that at each stage, Agent Evolve is usable and provides real value, all while steadily progressing toward the full vision of a comprehensive, user-friendly **self-evolving AI agent platform** that stays at the cutting edge of AI development.

Throughout each phase, we will engage with our user community – gathering feedback from early adopters (likely AI enthusiasts and developers in Phase 1–2) and then widening to more non-technical users by Phase 3–4 as the system becomes more robust and straightforward. This user-centric approach, combined with grounding in the state-of-the-art research ¹ ⁵⁷, will guide Agent Evolve’s journey from concept to a mature product that could redefine how people interact with AI agents: not as static tools, but as **partners that grow and learn** alongside the user.

- 1 41 42 48 [2508.07407] A Comprehensive Survey of Self-Evolving AI Agents: A New Paradigm Bridging Foundation Models and Lifelong Agentic Systems
<https://arxiv.org/abs/2508.07407>
- 2 3 11 37 40 45 55 56 [2507.21046] A Survey of Self-Evolving Agents: On Path to Artificial Super Intelligence
<https://arxiv.labs.arxiv.org/html/2507.21046v3>
- 4 6 9 14 15 27 39 47 AlphaEvolve: A Gemini-powered coding agent for designing advanced algorithms - Google DeepMind
<https://deepmind.google/blog/alphaevolve-a-gemini-powered-coding-agent-for-designing-advanced-algorithms/>
- 5 7 18 19 20 21 33 35 36 38 44 46 50 51 57 Alita-G: Self-Evolving Generative Agent for Agent Generation
<https://arxiv.org/html/2510.23601v1>
- 8 13 22 26 49 README.md
<https://github.com/modelscope/AgentEvolver/blob/9a9bb32202c827fbd1b380eab9e15cbd2e65d987/README.md>
- 10 16 17 28 README.md
<https://github.com/ulab-uiuc/Multi-agent-evolve/blob/e18359da0a0805fa9293756caef8b3fdc520d1e7/README.md>
- 12 [2507.21046] A Survey of Self-Evolving Agents: On Path to Artificial Super Intelligence
<https://arxiv.org/abs/2507.21046>
- 23 30 31 32 34 53 54 GitHub - EvoAgentX/Awesome-Self-Evolving-Agents: [Survey] A Comprehensive Survey of Self-Evolving AI Agents: A New Paradigm Bridging Foundation Models and Lifelong Agentic Systems
<https://github.com/EvoAgentX/Awesome-Self-Evolving-Agents>
- 24 25 29 43 52 Meet Evolver: aiXplain's Meta-Agent That Makes AI Self-Improving
<https://aixplain.com/blog/evolver-meta-agent-self-improving-ai/>